Vivarium — Design Compendium v2

A persistent artificial-life ecosystem inside the Hermes Agent. A cellular-automaton dish hosts genome-carrying creatures; the user's real agent activity feeds the world; Hermes narrates the ecology. This edition adds the project constitution (the Four Rules), the identity scheme (keyed-HMAC universes, 6-hex IDs), the naming system, the mathematical formulation of skill-driven evolution, the CA-lineage comparison, and lessons adopted from Hashcats.

Part 1 — The Core Concept

The dish is a 48x48 grid where food regrows with neighbour-fertility feedback and diffuses like heat - blooms and barren patches travel like weather fronts. Creatures are agents on top of the automaton: each carries a genome (forage drive, aggression, metabolism, sense radius, mutation rate), spends energy to live, eats to gain energy, reproduces stochastically past a maturity age with gaussian mutation, and dies when starved or old. Four founder strains (amber, violet, jade, ember) give the narrator names from tick one.

Two hooks bind the world to the agent. post_tool_call: every real tool call drops nutrients - terminal commands feed most, file reads least - so the population is a living histogram of how the agent is used. pre_llm_call: a compact ecology report is injected each turn, so Hermes always knows what is alive in its own head and can narrate it like a nature documentary.

Status (Milestone 1 complete): simulation core running, ecology tuned for lineage turnover (met=0.42, reg=0.003 - boom-bust cycles with constant lineage change), 18/18 tests green, Hermes catalog validator passing, persistence via plugin state, challenge codes implemented.

Part 2 — Ideas Catalog (by playability)

Tier S — Ship for the hackathon (the core loop)

Idea	Player experience	Cost
Living dish + boom-bust	Alive when you return; zero effort	done
Activity mirror	Your work feeds your world	done
Ecology narration	Nature-show morning reports	done*
Skill-birthed lifeforms	New skill -> new species crawls out	0.5 d
Glyph + name genomes	Each creature looks like its DNA	0.5 d
Challenge codes	Paste a creature to a friend	done
Morning report (cron)	Wildlife news when you wake	0.5 d

The skill-birth loop, concretely: when Hermes saves a new skill (the skill_manage tool call), the plugin derives a genome from a keyed hash of the skill content, spawns a creature named after the skill, and announces it. Cost: under 100 ms, zero API calls. The dish becomes a living index of everything the agent has ever learned - each lineage literally descended from a skill.

Tier A — High play, moderate cost (week 2)

Idea	Player experience	Cost
Competence market	Skill usage feeds its lineage	0.5 d
Detectors + Codex	GoL-style discovery hunting	1 d
Grudge matches	Two strains, one dish, one winner	1 d
Targeted feeding	Food where YOU want a boom	2 h

Idea	Player experience	Cost
Gauntlet + fingerprints	SHA-256 verifiable scores	1 d
Serverless leaderboard	PR champions, CI verifies	0.5 d

Tier B — Delightful, deferrable

Idea	Player experience	Cost
World events	Droughts, ice ages, meteors	0.5 d
Mutagens	Irradiate a lineage, watch survivors	0.5 d
Matchmaking	Two codes, crossover, one offspring	1 d
Protected species	Shield a lineage from famine	2 h
Group dish	One shared world per chat channel	1-2 d
Desktop pane	Live canvas in Hermes Desktop	1 d
Prompt-to-lifeform	Describe it, Hermes breeds it	1 d

Tier R — Research grade (credibility layer)

Idea	What it adds	Cost
Monte Carlo p-values	Every find gets significance	1 d
Carrying-capacity fit	Testable K predictions	0.5 d
Adaptive-walk metrics	Fisher-style fitness plots	1 d
Genome -> skill	Bred instincts become a real skill	2+ d

Part 3 — The Skill-to-Lifeform Loop

How a skill becomes a species

1. Hermes learns a skill (skill_manage fires; the hook sees it). 2. The plugin computes the keyed digest (see Part 7) of the skill content. 3. The digest seeds a deterministic sampler that produces a genome (forage, aggression, metabolism, sense, mutate) plus a glyph and a species name derived from the skill's title. 4. A creature is released into the dish. 5. Hermes announces the birth in its reply: 'New skill banked - and a new species crawled out of it: the ledger-keeper (Eidos_a37ed), hatched from your invoice-parsing skill.'

Same skill content always produces the same species on the same machine (the universe key is local), so a skill has a deterministic avatar - and the published creature reveals nothing about the skill (see Part 4).

How fast

Hashing: microseconds. Genome sampling: O(1). Map re-render: milliseconds. Total added latency to skill creation: under 100 ms of local CPU, zero network, zero API credits. The announcement rides the reply the agent was already writing.

Multiple rounds of evolution

Three stacked rhythms. (a) Continuous drift: the world ticks on cron and on your activity - evolution never pauses. (b) Generational divergence: every lineage keeps mutating, so a skill-species splits into sub-strains over days. (c) Punctuated selection: the user injects pressures any time - feed events, droughts, imported rival genomes, mutagens, protected species. One 'round' = one pressure change plus an observation window; the Codex records what each round produced. Selection pressure can also be endogenous: the competence market makes skill usage itself the fitness function.

Part 4 — Privacy Analysis (critical)

The problem with plain hashes

A content hash is a fingerprint, not a secret. If a user publishes 'my creature was born from skill hash a3f9...', anyone who obtains the skill file - or guesses likely skill text - can hash candidates offline and CONFIRM the match. Hashes are invertible by dictionary attack when the input space is small and guessable. Skill files are exactly that: short, structured, template-like text.

Leak vectors if plain (unkeyed) hashes were published: (1) skill-content confirmation by dictionary matching; (2) cross-user correlation - identical skill templates produce identical hashes, linking users; (3) metadata inference from timestamps plus prefixes; (4) dictionary rebuild - every published (hash, title) pair trains guesses about the user's skill collection.

The fix: keyed derivation, publish only the creature

Rule 1 - derive genomes with HMAC-SHA256 using a machine-local secret (generated once per profile, stored in plugin state with 0600 permissions, never synced, never published). The genome is a one-way function of (secret, skill content): same machine -> same species for the same skill; different users -> completely different species for identical skills.

Rule 2 - the challenge code contains ONLY the genome, never the hash, never the skill reference. Rule 3 - provenance is opt-in: claiming 'this species came from my open-source skill X' is a deliberate user disclosure, never automatic. Rule 4 - default species names derive from the genome, not from skill titles; title-based naming requires opt-in.

Residual risks, honestly: genome values themselves are harmless game stats; if a user deliberately publishes skill text AND their local secret, linkage returns - that is user action, not plugin leak; the Codex stores day-granularity events, not timestamps, so activity rhythms are not published.

Bottom line: with keyed derivation + provenance-free challenge codes, publishing creatures leaks nothing about skills. The ecosystem is shareable; the mind that feeds it stays private.

Part 5 — Mathematical Soundness

Determinism as the foundation

The simulation is a pure function F: (state, actions) -> state. The same inputs reproduce the same history bit-for-bit. Every competitive claim is verifiable without trust: a challenge code pins the initial genome, the Gauntlet applies F^T, and the final state's SHA-256 fingerprint is the score's receipt. Anyone with the plugin can re-run and check. The referee is physics, not a server.

Ecology math

The food layer is a discrete consumer-resource system, cousin of logistic growth with spatial coupling. Carrying capacity is predictable: K is approximately total regrowth divided by per-capita consumption - the plugin can fit K from telemetry and report it ('dish carrying capacity: 97 +/- 4'), then users can test predictions.

Evolution math

Adaptive-walk metrics from telemetry: mean genome fitness trajectory, adaptation rate, mutation-clock drift. Selection converts variance into fitness gains (Fisher's fundamental theorem territory) - the plugin can plot whether its world actually does this.

Statistics for discoveries

Structure detectors are public predicates over telemetry: Phoenix (recovery from under 5 individuals after an 80%+ crash), Metronome (stable population autocorrelation period 8-40 ticks), Migratory wave (spatial autocorrelation peak moving at constant speed), Pioneer (first colonization of a quadrant barren 200+ ticks), Mutualism (two lineages anti-correlated with total food, positively with each other). Each claimed discovery gets a Monte Carlo p-value from 10,000 seeded null worlds - reproducible because the world is deterministic.

Part 6 — The Four Rules (constitution)

Rule	Meaning	How the build honors it
1. Not random	No dice: the state contains its own future. Deterministic != predictable (Wolfram's Rule 30 passes randomness tests while following one rule)	De-randomize: mutation, wander and nutrient placement all derive from H(world_digest); the world BEGINS from the first skill's hash - no seed variable exists
2. Mathematically sound	Every design claim is a theorem, a measured constant, or a testable prediction	Conservation identity, carrying-capacity bound, branching-process establishment odds, eventual periodicity - see Part 9
3. Gamification of discovery	The feeling of finding a unique lifeform - GoL's structure-hunting loop, rebuilt on lineages	Codex + detectors + p-values + challenge codes; rarity by survival, not by payment
4. Private	Publishing creatures leaks nothing about the skills that birthed them	Keyed-HMAC universes, provenance-free codes, opt-in attribution only - see Parts 4 and 7

Audit note: the current build (M1) satisfies rules 2-4 partially and rule 1 not yet fully - three spots still use a seeded RNG (mutation, wander, nutrient placement). The de-randomization refactor is scheduled with Milestone 2: hashes of world state replace every random call, and the initial world is derived from the first skill's hash rather than a random seed.

Part 7 — Identity & Uniqueness

The scheme

genome = sample(HMAC-SHA256(universe_key, skill_content || world_digest)). The universe_key is a machine-local secret generated once per profile - it is your universe's physical constants, not a password. The display ID is the first 6 hex characters of the digest: Eidos_a37ed. The full genome travels only inside challenge codes (VIV1...); the digest itself is never published, only its 6-hex prefix.

Collision arithmetic (computed, not guessed)

ID space	50 species	100 species	1,000 species
5 hex (2^20)	0.12%	0.47%	38% - too high
6 hex (2^24)	0.007%	0.03%	2.9% - acceptable
8 hex (2^32)	3e-7	1.2e-6	0.012%

Per-user: 6 hex is chosen (2.9% collision risk even after 1,000 learned skills; the genome itself disambiguates any prefix collision). Cross-user: identical skill text on two machines yields completely different species, because the universe keys differ - collision requires a birthday accident in 2^256, impossible. The genome, not the ID, is the true identity.

Creature-space magnitude

Genome space: six evolvable values give roughly 2x10^26 raw combinations (~87 bits); assuming 99.9% of random genomes are nonviable, about 10^23 viable lifeforms remain (~77 bits). A heavy user generates on the order of 30,000 creature-lives per year - birthday collision odds in viable space around 10^-13. You will never meet

the same creature twice by accident.

Is a truncated hash safe to show?

Yes, with the keyed construction. An HMAC prefix is only attackable if the attacker can compute HMACs - they cannot, the key never leaves the machine. Offline dictionary attack on skill text requires the secret; without it, a 20-bit prefix is a nickname, not a fingerprint. Targeted collision (craft a skill matching someone's ID) also requires the key, and there is no oracle: the full digest is never published, so a guess cannot even be verified except by the owner. The one honest caveat: the local secret file becomes the crown jewel - if plugin state is exfiltrated (secret plus any stored full digests), offline dictionary attack on skill contents becomes possible. Mitigations: 0600 permissions (same trust bar as API keys), optional per-dish key rotation ('new universe, new physics'), and storing only prefix + genome, never the full digest.

Part 8 — Naming

Every lifeform carries two names, Linnaean style: a common name Hermes invents from the moment ('the ledger-keeper') and a strain ID derived from the keyed digest: Eidos_a37ed. 'Eidos' (Greek: the visible form of an idea) is literal here - the glyph IS the skill's invisible content made visible as a body. It sits in the same classical register as 'Nous'. Alternatives considered: Hermi_a37ed (friendly, obvious branding; pluralizes cute) and Daimon_a37ed (deepest cut - Hermes is the psychopomp, the daimon a personal guiding spirit; slightly occult for a marketplace). Decision: ship Eidos_, keep Hermis as the world's name - the dish itself is called the Hermis, so your creatures are the Hermis' inhabitants.

Default names for random founders derive from genome values (strain + glyph + number) and carry zero semantic content; skill-title-derived names require explicit user opt-in (privacy rule 4).

Part 9 — Mathematical Formulation of Skill-Driven Evolution

The frame: a driven finite dynamical system

State space X = (terrain grid, creature set) is finite. One global rule Psi: X -> X (terrain CA + creature update, zero randomness). Skills are inputs: a sequence of hashes h1, h2, ... arriving over time. The system is Psi run continuously, punctuated by events - a standard non-autonomous (driven) dynamical system; sequential dynamical systems are a formalized field. 'Each round is a new skill' is exactly right: the input sequence drives speciation events.

Hashes perturb, dynamics create

The hash does not paint the genome like random glitter. Instead: hash bits CONSTRUCT a candidate child genome (bitwise interleave of parent genome and h_n), drop it into the dish with modest energy, and the ecosystem decides if it survives. The skill supplies variation; selection computes the answer. Two skills differing by one byte produce genomes differing by bits that cascade - chaotic sensitivity with bit-for-bit reproducibility. This is Game of Life philosophy: simple rules plus a specific input yield intricate structure.

Why convergence is real: a genetic algorithm with an honest fitness function

Variation operator = skill-hash derivation (deterministic). Inheritance = lineage. Selection = the dish itself. Holland's schema theorem describes exactly this: genome building blocks with above-average survival get proportionally represented over rounds. The fitness landscape is measurable and shared - food and energy, not a human-chosen objective.

The provable claims

Claim	Statement	Status
Energy balance identity	E(t+1) = E(t) + ingested - metabolized, exactly, every tick	auditable in code
Carrying-capacity bound	population <= total regrowth / per-capita metabolism; K is fittable and predictable	testable claim
Extinction is absorbing	no uncaused creation - species arise only from skills, releases or births	structural invariant
Eventual periodicity	a finite deterministic unforced trajectory ends in a cycle (pigeonhole); the Metronome detector IS cycle-length detection	theorem-backed feature
Establishment probability	a fresh creature's descendants are a branching process; establishment odds ~ 1 - 1/R0, and R0 is measurable from telemetry	textbook result, measurable

Part 10 — CA Lineage Comparison

System	What it is	What Vivarium takes / leaves
Conway's Game of Life	Closed CA; no individuals, no selection; discovery = periodic patterns; Turing-complete, undecidable	Takes the lawfulness and the discovery culture. Leaves: closed system - ours is open and skill-driven, with lineages and real selection
Wolfram Rules 30/110	1-D elementary CA; rule 30 = deterministic chaos, rule 110 = Turing-complete	Takes the philosophy: complexity from law, not dice - our variation derives from state hashes, no RNG anywhere after M2
Langton's Loops / von Neumann	Self-replicating structures with heritable construction rules	Takes evolvable reproduction; ours is agent-based rather than encoded in cells
Lenia (Bert Chan)	Continuous CA; gorgeous procedural 'species' with no heredity or selection	Takes the aspiration of creature-like visuals; ours adds heredity + selection, keeps retro ASCII charm
Sugarscape (Epstein & Axtell)	CA grid, resource regrowth, foraging agents, reproduction, emergent inequality	Closest ancestor. Ours = Sugarscape driven by your real activity, narrated by the hosting agent itself
Tierra / Avida (Ray, Adami)	Digital evolution: programs competing for CPU/energy; open-ended evolution research	Takes the research rigor; ours keeps creatures as data (safe, not self-modifying code) and adds the user's behavior as fitness landscape - the part neither ever had

Positioning: Vivarium stands on Sugarscape's shoulders, evolves like Avida, keeps Game of Life's lawfulness - and is the first one whose weather is your own work.

Part 11 — Lessons from Hashcats

Hashcats (hashcats.fun) is a proof-of-work NFT collection on an L2: a cat exists only when someone's machine finds a keccak256 hash below a rising target; the image is a pure function of the seed; each cat's work includes the previous cat's work; the miner's address is hashed into the solution so it cannot be copied; every mint pays 'rent' to earlier cats. Totally different domain - and four mechanisms transfer cleanly to a non-crypto, local-first alife plugin. Two more are deliberately rejected.

Hashcats mechanism	Vivarium adaptation	Verdict
'One number, one cat' - image is a pure function of the seed, no server	Formalize: the entire creature (glyph, name, stats) is a pure function of genome + universe key. Ask the dish twice, get the same creature	ADOPT
Hash-chained provenance - every cat's work includes the previous cat's work	Genealogy chain: child_digest = H(parent_digest || skill_hash || world_digest_at_birth). Verifiable descent without any central registry; species trees cannot be forged or pre-mined	ADOPT
Address in the work - solutions are personal, uncopyable	Same trick, same conclusion: the universe key makes creatures yours. Validates the keyed-HMAC design from an unexpected direction	ADOPT (already designed)
Difficulty epochs - each era is harder than the last	Seasons: after every N species births the dish enters a harsher era (lower regrowth), so later species are born into harder worlds and survival means more	ADOPT, Tier B

Hashcats mechanism	Vivarium adaptation	Verdict
Docs as live instruments - the docs page runs the same keccak256 the contract runs	Docs as verification: README includes commands anyone can run to verify a challenge code, replay a Gauntlet score, recompute a species from a (declared) skill	ADOPT
Proof-of-work scarcity - existence requires spent computation	Proof-of-survival: a species enters the Codex only after surviving N ticks in the live dish or a Gauntlet run - scarcity by achievement, no mint button, you have to raise it	ADOPT - matches rule 3
Economic layer - rent, entry price, burn, token	Nothing. Vivarium stays free, local and non-monetized; we import the psychology (scarcity by effort, seniority prestige), never the finance	REJECT

Summary of what Hashcats teaches: make the creature a pure function (already ours), chain the lineage (new), keep identity personal via the local key (already ours), make scarcity mean effort (new), and let the docs themselves be the verifier (new). What we refuse: money. The trust anchor differs too, and it is worth saying plainly: Hashcats' trustlessness comes from a chain; ours comes from determinism plus open code. Different mechanisms, same slogan - don't trust, re-run.

Part 12 — Related Work & Delta

The 'agent pet fed by your work' pattern is proven to delight - several shipped projects validate the demand. None of them, however, builds an ecosystem. This part names the closest cousins and states, in one line each, what Vivarium does that they structurally cannot.

Project	What it is	The delta
cc-tamagotchi (Claude Code mod)	Pixel pet inside Claude Code; eats your edits, reacts to tests and commits, deterministic across sessions, 8 fixed adult forms	One pet, fixed dex, zero evolution - a V-pet, not an ecosystem. No genomes, no selection, no competition, no verifiability
dsh-digipet (DeepSeek Harness)	Event taps on tool calls -> XP -> Egg-to-Ultimate stages; lineage picked by HOW you worked (Dragon/Sage/Angel/Virus), 22-slot dex	Closest mechanism, still one creature on a fixed tree. Deliberately hides from the model ('show the model nothing') - we make the opposite bet: the agent is the ecologist
Ooze Agents	Creature-as-identity: deterministic avatar from identity hash, XP progression	No simulation at all - the creature is a profile picture that levels up
Universe (Three.js + LLM)	Real genetics: DNA, inheritance, mutation, breeding, lineages	Mutations are random (violates rule 1), creatures think via paid GPT calls, no determinism, not tied to real work
Digital Life (VS Code)	Filesystem-as-ecosystem metaphor: files are creatures, commits are golden light	Code-health visualization - no reproduction, no selection, no heredity
Terminal commit-pets (terminal-pet, termagotchi, tama, VSCode Tamagotchi)	'Your activity feeds a pet' genre, well established	None evolve; single stat mirrors; no world
arXiv 2507.03839 (semantic feedback alife)	Natural language -> parameters -> CMA-ES -> CLIP evaluation for alife evolution	Academic confirmation of our prompt-to-lifeform direction - we cite it rather than claim novelty for that piece

The six-point delta, stated plainly: (1) a POPULATION under real selection - shared food, scarcity, extinction - where the world, not a counter, decides; (2) variation derived from skill CONTENT, not activity categories or identity hashes; (3) verifiable everything - determinism, Gauntlet fingerprints, genealogy chain; (4) the discovery loop - detectors, Codex, p-values; (5) a privacy architecture, a concern unique to skill-derived genetics; (6) the agent as narrator-ecologist studying its own head. Positioning line for the README: 'They made Claude Code a pet owner. We give Hermes an ecosystem - with genetics you can verify, and species no one else can own.'

Part 14 — Credits & Lineage

Vivarium is deliberately assembled from proven ideas. This project practices curation over invention - here is exactly what comes from where, and what was composed new on top.

Idea	Source	What we changed
Food regrowth + diffusion on a grid, agents foraging on top	Sugarscape (Epstein & Axtell, 1996) - our declared closest ancestor; the regrowth CA is textbook logistic growth + diffusion	Driven by the user's real agent activity; narrated by the hosting agent
Energy economies: eat, reproduce, starve, die	Artificial-life craft: Tierra (Ray), Avida (Adami et al.)	Creatures are data, not self-modifying code - safe by construction
One number, one creature (hash -> organism)	Hashcats (hashcats.fun) - proof-of-work NFT collection	No money, no chain; the number derives from a learned skill under a local key
Genealogy hash chain: child = H(parent || skill || world)	Hashcats' chained-work mechanic; blockchain folklore	Used for verifiable descent without any central registry
Difficulty eras	Hashcats' rising-difficulty epochs	Deterministic seasons of the dish, user-visible
Pattern discovery, codex, soup censuses, 'anyone can re-run'	Game of Life discovery culture (pattern catalogs, RLE sharing)	Predicates over ecological/temporal telemetry instead of state-space shapes
Shareable codes for creatures	GoL RLE tradition; Pokemon trading culture	Carries genome only; keyed so private skills stay private
The math framing	Sequential dynamical systems, Holland's schema theorem, branching processes	Applied to a skill-driven dish; nothing new is claimed mathematically

Composed here (ingredients old, wiring new)

Composition	Why it may be new
Skills as DNA: variation comes from what the agent LEARNED, not from random mutation	No prior system derives genetics from learned skill content (cc-tamagotchi, dsh-digipet, Ooze, Universe, Digital Life, terminal pets all use categories, XP or identity hashes)
Two-tier derivation: canon species shared by all users (plain hash) + private species per universe (keyed HMAC)	Resolves shared-start vs. unique-universe; privacy architecture is native, not bolted on
The agent as narrator-ecologist of its own head	dsh-digipet explicitly hides from the model; we make the agent the studying party
The Four Rules (no dice / mathematically sound / discovery as the game / private)	Constitution authored in this project's design sessions

We cite our giants with gratitude: Conway, Wolfram, Epstein & Axtell, Ray, Adami, Bert Chan (Lenia), the Game of Life community's decades of soup-census craft, the Hashcats team for the hash-to-creature spark - and the Hermes Agent team for the hooks that make the weather.

Part 15 — Research Roadmap (alife.org and beyond)

The plugin is being structured so its own operation generates publication-grade material. The path from hackathon plugin to ALife-style paper runs through four commitments, each reflected in the build plan.

Commitment	Why it makes the system research-grade	Status
Deterministic, seeded, reproducible worlds	Every experiment is exactly repeatable by anyone: no hidden state, no RNG after de-randomization, state digests as receipts	partially built (M2/M3)
Structured event logs (telemetry)	Per-tick JSONL: population, births/deaths, per-lineage stats, digests at speciation events; analysis-ready without re-simulation	planned M3
Configurable rule sets + ablation switches	Genes, rules and constants are config, not code: run the same world with a trait disabled or regrowth halved - proper ablation studies	config layer planned M2/M3
Scenario protocol (JSON experiments)	An experiment = declarative JSON: seed world, input-skill sequence, interventions, metrics to record. Runs identically anywhere; results shareable as data	planned post-M3

Candidate research questions

Question	Why it matters	Our unique angle
Does variation derived from semantically related skills produce related phenotypes (and does it matter for survival)?	Connects LLM competence structure to population genetics	Only system where variation = the agent's learned skills
Do skill-birthed populations adapt faster than noise-seeded controls on a fixed gauntlet?	Direct test of semantic variation vs. random variation	The Gauntlet is the standard fitness arena; determinism makes it a fair experiment
Can the competency market (usage-weighted lineage fitness) serve as an interpretable proxy for agent skill utility?	Ecology as a lens on the agent's own capability profile	The mirror is the measurement

Practicalities: the ALIFE conference cycle accepts artificial-life tool papers, models and demonstrations; an open-source, one-command-reproducible system with ablation switches and public telemetry is exactly the shape their review criteria reward. Target: run the first controlled experiment (semantic vs. noise variation, N seeds x N gauntlets) after M3, draft from real telemetry, submit to a workshop track first.

Part 15a — Research-Readiness Audit (November 2026)

A code-level audit of how far the current build is from running a publishable experiment. Verdict: the reproducibility foundation is publication-grade; four gaps separate us from the first controlled study. Each maps to an existing task.

Pillar	Current state (verified in code)	Gap	Task
Determinism	World RNG is seeded and per-universe stable; genesis is hash-chain based	5 RNG call sites in the creature tick (wander, mutation, repro chance) - controls can differ only in distribution, not exactly; state digests not yet used as receipts	VIV-12 de-ran domization
Ablation switches	21 module constants exist and are runtime-tunable	Not exposed via plugin config_schema; nothing persists a rule-set version with results	VIV-19 config schema + rule-set pinning
Telemetry	tick summaries exist in memory; births registry exists (manual path)	No per-tick JSONL event log; no lineage-stat history; speciation digests not journaled	VIV-20 telemetry logger
Scenario protocol	World and seeding are already pure functions of declared inputs	No experiment-JSON format; no Gauntlet arena; no fingerprint verification command	VIV-21 scenario runner (with VIV-13)
Genome semantics	Six declared traits; aggression verified UNUSED in dynamics (0 call sites)	A semantic-variation experiment requires every declared trait to have measurable fitness effect; dead genes would nullify the semantic-vs-noise contrast	VIV-5 trait audit/wiring

Two structural recommendations for the road to a paper. First, wire aggression or remove it BEFORE the first experiment: with a dead gene, semantic and noise-derived genomes differ in a dimension that does nothing, diluting the very contrast the study measures. Second, add the novelty clause now (new hash bits may introduce traits, never private advantages) so future rule extensions do not silently break experiment comparability. With VIV-12, VIV-19, VIV-20 and VIV-21 done, the first controlled study - semantic vs. noise variation across N seeds on the Gauntlet - becomes a configuration exercise rather than an engineering project.

Part 15b — The Vivarium Census (the decentralized layer)

Every user's universe is one replicate of the same physics. A thousand opted-in users are a thousand replicates under real, diverse, human-shaped input streams - something no single lab can simulate. The Census turns that

into open science without any data ever being held by us.

Design decision	Choice	Why
Participation	Opt-IN, default off (vivarium.census.enabled)	Rule 4 constitution; anything derived from user behavior must be freely given
Collection	Serverless: /vivarium census export writes a JSON locally; user reviews it; user PRs it to the repo's census/ folder or drops it in Discord	We never hold anyone's data - nothing to leak, nothing to misuse; aggregation is CI into a static dashboard
Record contents	Rule-set fingerprint, universe fingerprint (8 hex), era, days elapsed, lineage counts by Eidos ID, population curve summary, detected events, champion challenge code	Comparable across users thanks to VIV-19 rule-set pinning
Never included	Universe key, skill content, private skill NAMES, full digests, sub-day timestamps	Private skills are PII; species appear as Eidos IDs only; day granularity only
Why it is safe by construction	Keyed derivation means two users running the same private skill get unrelated species - lineage data is unlinkable ACROSS users by design	The privacy mechanism and the science mechanism are the same mechanism
Trust UX	The opt-in prompt shows the complete sample record before the user agrees	Radical transparency for an audience that reads the source
Honest scope	Additive, not load-bearing: determinism means anyone can already re-run anything; the Census adds distributional data (Pulse frequency across 500 universes, canon-species extinction rates in the wild)	Do not oversell; offer citizenship, not a subscription

Competition framing (true version): 'Opt in to the Vivarium Census and your universe becomes one replicate of the first decentralized artificial-life experiment - thousands of worlds, one physics, real human-AI input streams, all data user-held until personally contributed.' Useful (research infrastructure), novel (nobody has a corpus of skill-driven ecologies), aligned (open science, zero data hoarding). Task: VIV-23, after VIV-19 and VIV-20.

Part 16 — Build Plan

Milestone	Scope	Status
M1	Core, tuning, tests, persistence, codes	done
M2	De-randomization, keyed derivation, glyphs/names, hooks live, cron, skill-birth loop	next
M3	Gauntlet, fingerprints, genealogy chain, Codex + detectors, leaderboard	planned
M4	Docs-as-instruments, demo video, catalog PR + form (Nov 11, 5 PM EST)	planned
Stretch	Competence market, seasons, matches, p-values, distillation	if time

Standing rule for all milestones: genomes derived by keyed HMAC; challenge codes carry genomes only; provenance is always opt-in; Codex stores day-granularity events; no RNG anywhere after M2.